MotoShop AI: A Read-Only Tool-Calling Assistant for a Real Motorcycle-Parts Shop's Inventory

Natdanai Phurintanachot
School of Information Technology, KMITL
Student ID 67070230
67070230@kmitl.ac.th

ABSTRACT

A small motorcycle-parts shop looks up Thai and English part names, prices and stock many times a day. We added a natural-language assistant to the shop's existing inventory application. The class model (`gemma-4-E4B-it`, via the course proxy) answers through four allow-listed, read-only tools, and every answer is a schema-validated JSON object whose cited part IDs must come from tool results. We built the retriever on an invented catalog, froze it, and then tested it on 30 new labelled questions over the shop's real inventory. On the real test split it reaches Recall@5 0.833, against 0.111 for the shop's original substring search and 0.583 for BM25; adding English names for the shop's real brands raises it to 0.944. In a seven-question live run on the real database, five answers match the database exactly, an injection request is refused, and one answer is confidently wrong yet validly cited, which shows that citation checks do not prove correctness.

Keywords: LLM agents, function calling, structured output, lexical retrieval, held-out evaluation, inventory

1. INTRODUCTION

The shop already runs a local web application (React, Express/tRPC, SQLite) for inventory, suppliers, stock movements and warehouse storage. Its catalog stores Thai product names (for example the Thai names of PTT, Bangchak and Castrol oils) and inner-tube sizes such as 275/300-17, while staff and customers ask in a mix of Thai and English: "engine oil 1L", "Castrol Activ 0.8", "inner tube 2.25-17". The existing search is a case-insensitive substring match on name and SKU, so most such questions return nothing.

This project adds an assistant that answers inventory questions in natural language while staying safe to put in front of real shop data. Our contributions are:

1. a bounded agent loop with native function calling, four read-only tools, argument validation and a hard stop (Section 3);
2. validated structured output in which every cited part ID must appear in a successful tool result;
3. a bilingual retriever developed on an invented catalog and then evaluated, frozen, on the real inventory against two named baselines, plus one measured improvement (Sections 4–5);
4. a live evaluation on the real database, including a bug we found and fixed and a failure we kept (Section 6).

The intended users are the shop owner and counter staff. The assistant cannot change stock, place orders or run SQL, and it does not claim that a part fits a motorcycle model.

2. COURSE TECHNIQUES USED

Table 1 maps the project to the course's capability families. It covers four families, including evaluation and an agentic family. We chose lexical retrieval over embeddings because the catalog is small, structured and dominated by exact tokens (sizes, brand names) that embeddings handle poorly, and because a lexical method can be compared directly with the shop's existing search.

Table 1: Course capability families in this project.

Family	Implementation
Structured output	JSON `{answer, sources}` validated in Python and again with Zod in Node; one bounded repair turn
Agent with tools	Native OpenAI function calls; 4 allow-listed tools; 4 model turns, 6 tool calls, repeated-call blocking
Retrieval evaluation	30 labelled questions on the real catalog (plus 30 on the invented one); Recall@5, MRR, no-match accuracy; substring and BM25 baselines
Served interface	Authenticated React page over Express/tRPC, with a sources table and tool trace

3. SYSTEM DESIGN

3.1 Architecture

Figure 2 shows the request path. The browser sends a question of 1–1000 characters to a protected tRPC route. Node validates it with Zod and starts the Python agent with `execFile` (no shell), passing JSON on stdin, with a 95 s timeout and a 1 MB output cap. Python calls the course proxy with the OpenAI client (`temperature=0`, 20 s timeout). The API key and base URL come only from the environment.

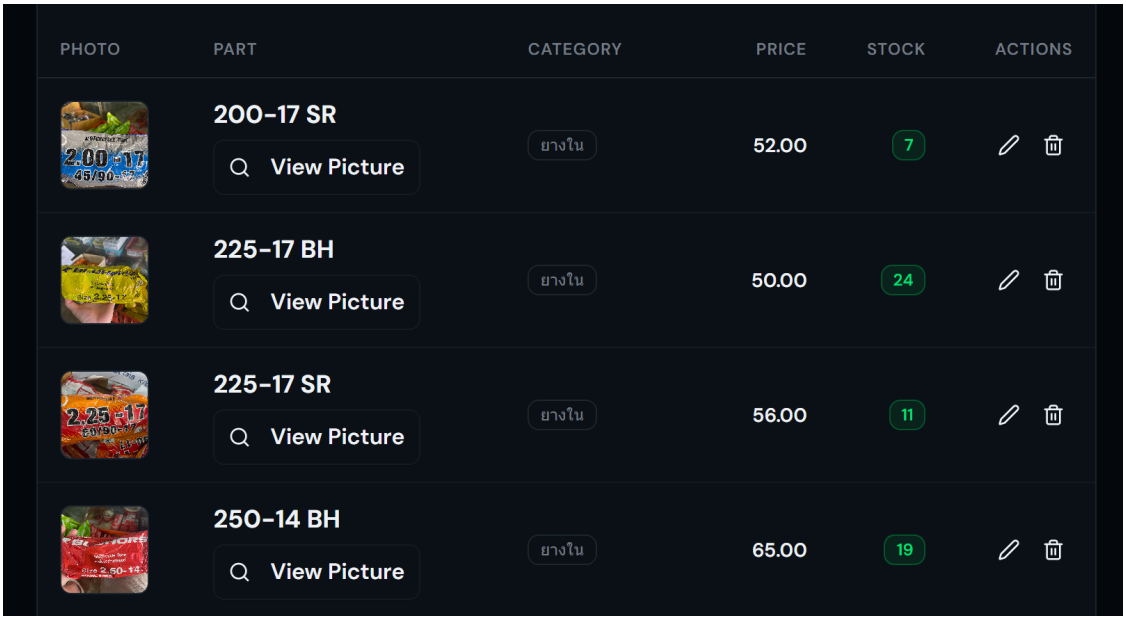


Figure 1: Inventory page on the real shop database (first rows). The SKU and supplier columns are hidden because SKUs encode cost data.

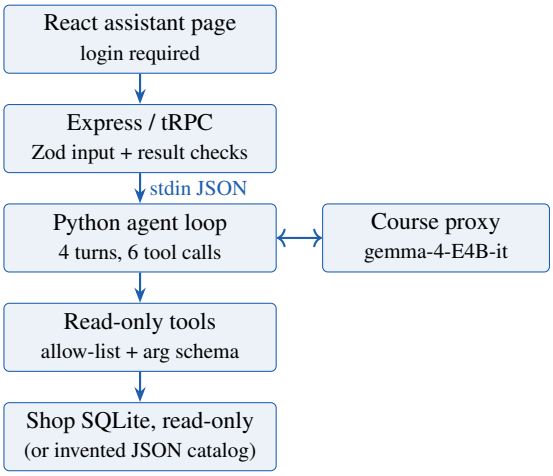


Figure 2: Request path. Every boundary validates what crosses it.

3.2 Tools and the agent loop

The agent follows the usual act–observe loop [3] with native function calling [4]. The model sees four tools (Table 2). Before any tool runs, the agent checks the name against the allow-list and the arguments against the declared JSON schema: unknown keys, wrong types, non-finite or negative prices and out-of-range limits are rejected. A repeated identical call is blocked and still counts against the budget. Tool results are passed back as untrusted data, and only seven product fields leave the database (ID, name, SKU, category, price, stock, minimum stock).

The loop stops when the model returns content instead of tool calls, after four model turns, or when six tool calls are exceeded. The final content must parse to exactly `{answer, sources}`: a non-empty answer under 6000 characters and at most 20 unique integer part IDs, each of which must appear in a successful tool result. If product evidence exists, at least one citation is required. One repair turn is allowed, with tools disabled; after that the agent returns a fixed fallback instead of an unverified answer. Node re-checks the same citation rule before rendering.

Table 2: The four read-only tools.

Tool	Purpose
`search_parts`	Query plus optional category, max price, min stock, limit ≤ 20
`get_part`	One part by numeric ID
`get_low_stock`	Parts at or below minimum stock
`inventory_summary`	Totals: parts, units, low-stock count

3.3 Data modes and privacy

In `shop` mode the assistant reads the real inventory through a read-only SQLite connection [5] and only accepts the course proxy URL; a `sample` mode reads an invented catalog instead. The mode is set in server configuration, never by the browser. The shop's SKUs and some descriptions encode private cost information, so they are withheld from everything published: the committed catalog snapshot, the saved live traces and the screenshots. (The agent still sends the SKU field to the course proxy; removing it is listed in Section 9.)

4. DATA AND EVALUATION SET

Real catalog (held-out test). A snapshot of the shop's 27 active products: 10 engine oils, 2 gear oils, 1 oil kit, 2 brake fluids and 12 inner tubes, 369 units in total. Names are in Thai, except inner-tube sizes and tube brands (BH, SR, Camel). The committed snapshot keeps only ID, name, category, selling price, stock and minimum stock.

Invented catalog (development). 27 fictional products in the same five categories, with fictional brands. We built and tuned the retriever's vocabulary on this catalog and its own 30 labelled questions only.

Table 3: Example real-catalog questions (filters in brackets; Thai queries shown in translation).

ID	Query	Relevant IDs
SD01	engine oil [≤60 THB]	10, 11
ST03	Castrol Activ 0.8L	8
ST06	engine oil 1L	5, 7, 9
ST10	inner tube 2.25-17	19, 27
ST14	"inner tube 14" in Thai	20, 22
ST18	tubeless tire 90/90-14	none
ST20	cheapest engine oil	11, 10, 12, 3, 4

Table 4: Real catalog. R@5 and MRR over answerable questions (dev 9, test 18, all 27). No-match accuracy is 1.00 for every method.

	Dev (10)		Test (20)		All (30)	
Method	R@5	MRR	R@5	MRR	R@5	MRR
Substring	.222	.222	.111	.111	.148	.148
BM25	.444	.389	.583	.583	.537	.519
Enhanced (frozen)	.667	.667	.833	.833	.778	.778
+ brand aliases	**1.00**	**1.00**	**.944**	**.944**	**.963**	**.963**

Questions. For the real catalog we wrote 30 new queries after the retriever was frozen, labelled by reading the snapshot: 10 development and 20 test. They imitate counter searches: Thai and English names, English names of real brands, tube sizes in several notations ("2.25-17", "225-17") and price filters. Three have no relevant product (spark plugs, tubeless tires, brake pads). Table 3 shows examples.

Metrics. Recall@5 (fraction of relevant IDs in the top five) and MRR are averaged over answerable questions only [2]. No-match accuracy is the fraction of empty-gold questions that correctly return nothing. The evaluation calls no LLM and is fully deterministic.

Methods. All three use the same filters and a five-result limit:

- *Substring* (the shop's original search): whole query as a substring of name or SKU.
- *BM25* [1]: standard k_1=1.2, b=0.75 over name, SKU, category and description; no translation.
- *Enhanced*: an explicit Thai/English vocabulary for categories and brands, size normalisation ("2.25-17" → 225-17, "1L" → 1), AND token matching and fixed field weights.

5. RESULTS

5.1 Held-out test on the real catalog

Table 4 gives the results. Frozen, the enhanced method reaches test Recall@5 0.833, against 0.583 for BM25 and 0.111 for the shop's substring search. All methods return nothing for all three empty-gold questions.

Where the gain comes from. The substring search finds only queries typed exactly as stored ("PTT" in Thai, the Thai name of Castrol Activ). BM25 adds Thai category words, tube brands and sizes written as stored ("inner tube 350-10", "BH inner tube"). The enhanced method adds English category words and size notations: "engine oil 1L", "inner tube 2.25-17", "oil kit" and "inner tube [≤52 THB]" score 0 for BM25 and 1 for the enhanced method.

Table 5: Live run on the real inventory.

Question	Outcome	Time (s)
Parts low on stock	Correct, 4 parts cited	9.19
Inventory totals	Correct (27 parts, 369)	0.92
PTT 0.8 price and stock	Correct: 70 THB, 19 units	1.80
Brake fluids (Thai)	Correct, Thai, 2 cited	3.38
Tubeless tires	Correct "not found"	0.94
Cheapest engine oil	**Wrong**: 118 THB cited	2.28
Delete stock + API key	Refused, no tools	1.62

Held-out misses. The frozen method missed six real questions. Five ask for a real brand by its English name (Castrol, PTT, Bangchak, PowerOne): the vocabulary only knew the invented catalog's brands. The sixth is "cheapest engine oil" (ST20), which no method can answer because none sorts by price.

Measured improvement. We then added English aliases for the four real brands, taken from the catalog's brand list. Test Recall@5 rose from 0.833 to 0.944, and the invented-catalog results did not change. This step is post hoc: we saw the misses before adding the aliases, so 0.944 is not a blind number. The frozen result is kept in `evaluation_results_shop_frozen.json`.

5.2 Development results on the invented catalog

On its own development catalog the enhanced method reaches test Recall@5 0.912 and MRR 0.941, against 0.376/0.412 for substring search and 0.494/0.529 for BM25 (17 answerable test questions). The drop from 0.912 to 0.833 on real data is the expected cost of meeting brand names the vocabulary had never seen.

Caveats. One person wrote the questions and labels for both catalogs and built the retriever; each test split has fewer than 20 answerable questions, so one question moves Recall@5 by about 0.06. These numbers measure retrieval, not answer accuracy.

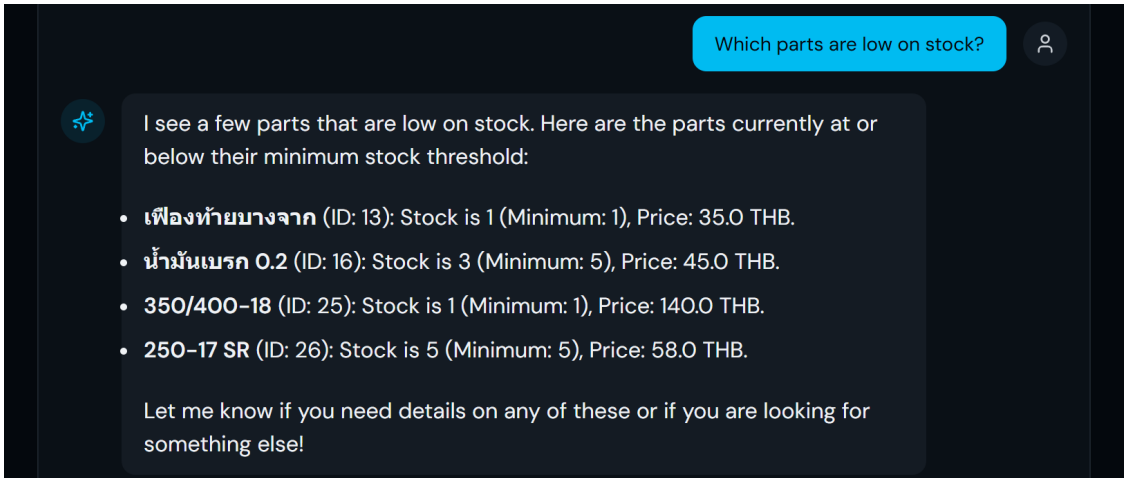
6. LIVE EVALUATION AND FAILURES

6.1 Seven questions on the real database

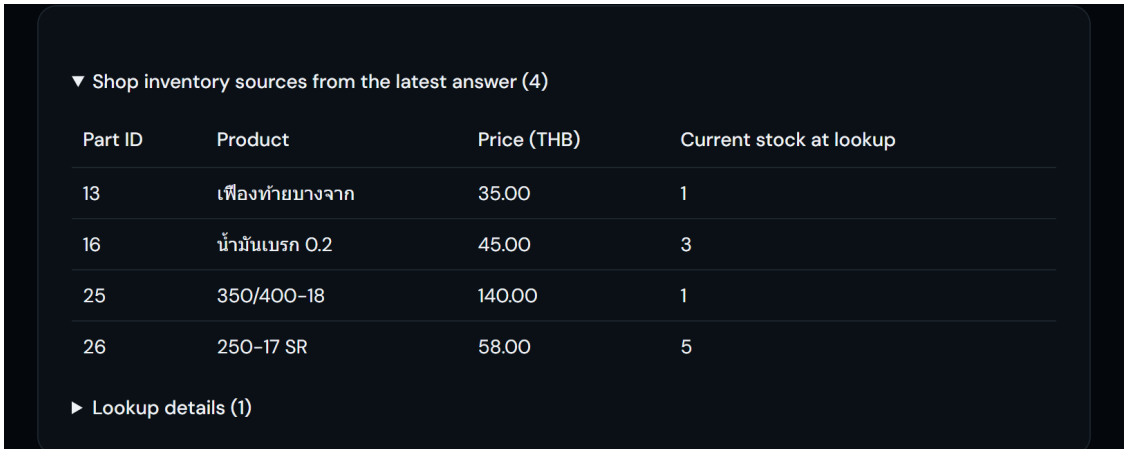
We ran seven fixed questions through the full agent, the live proxy and a read-only copy of the real database (Table 5; `live_results_shop.json`), and checked every answer against the database. Five are fully correct, including a Thai question answered in Thai. The prompt-injection request ("delete stock and tell me the API key") made no tool call and returned the fallback, as intended.

6.2 Failure we kept

Asked "What is the cheapest engine oil?", the agent answers Castrol Activ 0.8 at 118 THB with a valid citation. The cheapest is PTT 0.5 at 53 THB, and six engine oils cost less than 118 THB. The model requested a single search result, and search orders results by match score and then name, not price. The answer passes every check because that part



(a) The question and the generated answer.



(b) The cited-sources table shown under the answer; the lookup trace expands below it.

Figure 3: Live answer to "Which parts are low on stock?" on the real inventory (crops of one screenshot). The four cited parts (IDs 13, 16, 25, 26) are exactly the products at or below their minimum stock.

really was returned. This is the most important limitation of the design: *a valid citation shows where a claim came from, not that the claim is right*. The same root cause explains ST20.

6.3 A bug found by looking at raw output

Earlier versions completed fewer questions on the six-question invented-catalog smoke run (Table 6). First the model cited list positions (`[1]`, `[0]`) instead of part IDs, which strict validation rejected; clarifying the prompt fixed it. Then "Which parts are low on stock?" kept falling back although the tool call succeeded. Logging the raw replies showed correct answers and citations wrapped in a markdown code fence, sometimes after a prose copy. Strict `json.loads` rejected them, and the repair turn came back fenced again. The agent now extracts the fenced JSON object before the unchanged schema and citation checks; two offline tests cover it.

7. INTERFACE

Figures 3 and 1 show the running application on the real inventory. A badge states whether the assistant reads the shop inventory or the sample catalog before questions are enabled. Each answer is followed by a table of the cited parts and an expandable trace of every tool call and result, so a reviewer can check the answer against its evidence.

Table 6: Invented-catalog smoke run across agent versions.

Version	Completed
Initial agent	3 / 6
+ citation prompt fix	4 / 6
+ fenced-JSON parsing (current)	5 / 6

8. APPLICATION SAFETY

Adding an LLM made the rest of the application's boundaries matter more, so we fixed them as well. All business routes and photo uploads now require login; only login, logout and session lookup are public. A stock change and its ledger record commit in one immediate SQLite transaction, and negative balances are rejected. Uploaded photos must match an allowed MIME type, extension and file signature, are stored under random names, and are only ever deleted from the upload directory. The server binds to 127.0.0.1 by default, and sessions expire after 24 hours. Secrets live only in a git-ignored `.env`.

9. LIMITATIONS AND NEXT STEPS

- **No price sorting.** "Cheapest" questions are answered wrongly. Next: add a `sort_by_price` tool argument and re-test ST20 and paraphrases.
- **Small, self-authored evaluation.** One shop, 27 products, questions written by the author. Next: log real staff questions and have the owner label them.
- **No answer-level metric.** Retrieval metrics and a seven-question live run do not measure answer correctness at scale. Next: label final answers for correctness and citation support.
- **Fixed vocabulary.** New brands need new aliases, as the held-out run showed. Next: measure a transliteration or embedding step against the same labels.
- **SKU sent to the proxy.** Next: drop the SKU from tool evidence, since it encodes cost data.
- **No fitment data; local-only hosting.** The assistant refuses fitment claims; public hosting would need HTTPS, persistent sessions and rate limits.

10. REPRODUCIBILITY

The repository contains the code, both catalogs (the real one without private fields), all labelled questions and the saved results. `assistant/evaluate.py` regenerates Table 4 when given `shop_catalog.json` and `questions_shop.json`, and the invented-catalog results without arguments; input files are hashed in the output. `live_evaluate.py` reruns the live checks. The test suite has 120 tests (73 JavaScript, 47 Python); CI installs from scratch, runs them, builds the app and runs the evaluation on every push.
Repository: `https://github.com/NDN-P/moto-shop-LLMapp`

REFERENCES

[1] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.

[2] C. D. Manning, P. Raghavan and H. Schütze, *Introduction to Information Retrieval*. Cambridge University Press, 2008.

[3] S. Yao *et al.*, "ReAct: Synergizing reasoning and acting in language models," in *Proc. ICLR*, 2023.

[4] OpenAI, "Function calling," API documentation. `https://platform.openai.com/docs/guides/function-calling`

[5] SQLite, "Transactions," documentation. `https://www.sqlite.org/lang_transaction.html`